

MaiAgent Technical Documentation

Table of Contents

1. 生成 AI クイックスタート

- 1.1 大規模言語モデル（LLM）
- 1.2 RAG ナレッジベース検索システム
- 1.3 Embedding モデル
- 1.4 Reranker モデル
- 1.5 Parser 解析ツール
- 1.6 画像認識のサポート

2. MaiAgent Work | 従業員の生産性向上のための連携

- 2.1 自社システムへの MaiGPT の追加
- 2.2 MaiGPT 連携：適切なアシスタントの選択と機能の補完
- 2.3 メンバーのメモリ管理 API

3. MaiAgent Serve | カスタマーサービス連携

- 3.1 ソフトウェアベンダー向け連携ガイド：顧客向けの AI Agent の埋め込み
- 3.2 Web Chat の埋め込みと SDK
- 3.3 MaiGPT モードでの埋め込み
- 3.4 ページの Context の注入
- 3.5 LINE LIFF：LINE LIFF とは
- 3.6 LINE LIFF：連携方法
- 3.7 連絡先（Contact）の概要と連携
- 3.8 連絡先の ID 同期とトークン更新
- 3.9 ロール（Role）と連絡先（Contact）の違い
- 3.10 外部カスタマーサービスシステムとの連携（有人対応への引き継ぎを含む）
- 3.11 会話の閲覧権限とリアルタイムイベント
- 3.12 会話の引き継ぎと継続 API
- 3.13 Webhook

4. MaiAgent Build | AI アシスタントモジュール

- 4.1 システムプロンプト
- 4.2 ナレッジベース
- 4.3 FAQ 管理
- 4.4 応答評価とモニタリング結果
- 4.5 AWS Guardrails

5. MaiAgent Build | 高度な生成 AI 技術

- 5.1 Text to SQL
- 5.2 Function Calling
- 5.3 スキルモジュール
- 5.4 Canvas
- 5.5 AI セキュリティ保護の仕組み
- 5.6 MCP (Model Context Protocol)
- 5.7 OAuth 2.0 連携とトークンの自動更新の仕組み
- 5.8 ICAP コンテンツ検証連携

6. MaiAgent Build | API リファレンス

- 6.1 クイックスタート
- 6.2 AI アシスタント一覧
- 6.3 会話とメッセージ応答 (ストリーミング／同期)
- 6.4 会話とメッセージの作成
- 6.5 署名付き URL によるファイルアップロード方式
- 6.6 会話の添付ファイル管理 API
- 6.7 会話の生成物一覧 API
- 6.8 ナレッジベースの利用—作成と対話
- 6.9 API ツールエンジニアリングガイド
- 6.10 セルフホスト MCP Server の接続仕様
- 6.11 サイトのブランド設定 API
- 6.12 Credits 項目の表示順序
- 6.13 組み込みワークフロー応答モードの廃止

7. MaiAgent Build | AI Gateway

- 7.1 AI Gateway : OpenAI 互換 API 連携
- 7.2 AI Gateway のセキュリティ

8. MaiAgent Build | 外部サービス連携

- 8.1 Remote MCP サービスの概要
- 8.2 Composio 連携
- 8.3 Zapier 連携
- 8.4 Google Sheet 連携
- 8.5 n8n 連携

9. ID とアクセス権限

- 9.1 メンバーのプロビジョニング：SCIM／招待／SSO
- 9.2 リアルタイム同期：ユーザーイベントからの呼び出し
- 9.3 名簿同期ツール：CSV／JSON／データベース
- 9.4 SAML SSO シングルサインオン連携
- 9.5 OAuth リフレッシュトークンの失効とローテーション
- 9.6 ナレッジ管理権限（Query Metadata／クエリメタデータ）の概要
- 9.7 構築を開始—「クエリビルダー」の利用
- 9.8 構築を開始—JSON 形式の利用

10. プラットフォームの構築とデプロイ

- 10.1 概要
- 10.2 プラットフォーム環境（SaaS／プライベートクラウド／オンプレミス）
- 10.3 デプロイアーキテクチャ
- 10.4 クラウドモデル推論 API サービス
- 10.5 GPU 計算リソースのハードウェア計画
- 10.6 Celery の定期タスク設定と OAuth トークン更新
- 10.7 ファイルライブラリでの Office オンライン編集（ONLYOFFICE Docs 連携）

11. システムアップデート

- 11.1 ナレッジベースのファイル作成とメタデータ処理の最適化

生成 AI クイックスタート

大規模言語モデル（LLM）

選択時の重要ポイント

大規模言語モデルを選択する際は、以下のいくつかの重要な要素を考慮してください。

環境（Environment）：利用環境がインターネットに接続できるかどうかによって、クラウドモデルかオンプレミスモデルかを判断します。

品質（Quality）：モデルが回答結果を生成する能力と、指示への忠実さです。

速度（Speed）：テキスト生成の速度およびレイテンシに対する要件で、モデルの応答速度を確保します。

価格（Pricing）：モデルの利用コストを考慮し、利用ニーズに応じて適切なモデルを選択します。
（MaiAgent ではモデルの価格を考慮する必要はありません）

その他：マルチモーダルに対応しているか、Function calling に対応しているかです。

大規模言語モデル分析 - [Artificial Analysis](#)

MaiAgent で対応している大規模言語モデル

クローズドソースモデル

モデル名	説明	推論モデルかどうか	利用シーン
o4-mini	o3-mini-high より速度が速く、品質は o3-mini-high にやや劣ります	はい	高品質かつ高速を重視する選択肢
o3-mini-high	品質が高く、速度は中程度で、回答前に思考の連鎖（Chain of Thought）モードで多層的な思考演算を行い、より完全で精緻な回答を提供します	はい	難易度が高く、深い推論や創造力を要求するタスク
o3-mini-medium	速度が速く、品質は中程度です	はい	大部分のビジネス用途、シンプルな創作や一般的な質疑応答
o3-mini-low	速度が最も速く 、品質は基本的で、深い推論には欠けます	はい	スピードを求め、生成の深さを求めないシンプルなタスクに適しています

モデル名	説明	推論モデルかどうか	利用シーン
o1-mini 2024-09-12	o1 系列の大規模言語モデルは強化学習によって訓練され、複雑な推論を実行します。o1 モデルは回答前にまず思考し、ユーザーに応答する前に非常に長い内的な思考の連鎖を生成します。速度は最も遅く、 品質は優れています 。	はい	非常に難しい問題で、他の LLM ではどうにもならないとき
GPT-4o 2024-08-06	品質、速度ともに平均的に高いです	いいえ	指示への忠実さと論理能力は Claude 3.5 Sonnet にやや劣りますが、速度は Claude 3.5 Sonnet より速いです。よく使われる選択肢です👍
GPT-4o mini 2024-07-18	速度が速く、品質は中程度です。品質は Gemini 2.0 Flash よりやや低いです	いいえ	シンプルなタスク、Gemini 2.0 Flash を選択できない場合の代替品
Claude 4 Sonnet	速度はやや遅めで、 構造化データの生成／抽出能力が高く 、さらに ツール呼び出し (Tool Calling) が特に得意です。論理推論とコーディングの性能は Claude 3.7 Sonnet を上回り、ハルシネーション率はさらに低下しています。	ハイブリッド推論モデル	Agent モードの第一候補👍 高難度のタスク、専門領域での応用、超長文の対話に適しています。
Claude 3.7 Sonnet	速度はやや遅めで、 構造化 (Structured) データの生成が得意 で、Claude 3.5 Sonnet より強い論理推論能力を備えています。 ハルシネーションの発生率が低い です	ハイブリッド推論モデル	大部分のケースで第一候補👍 複雑度の高いタスク、専門領域、長文対話の用途に適しています
Claude 3.5 Sonnet	役割指示に忠実で、論理推論能力は Claude 3.7 Sonnet に比べて弱いものの、 速度は比較的速い です。 ハルシネーションの発生率が低い です	いいえ	速度が遅いと感じた場合は Gemini 2.0 Flash に切り替えられます
Gemini 2.5 Pro	長文の対話やプログラム生成のケースでは品質が Claude 3.7 Sonnet より優れていますが、Agent モードやツール呼び出しでの性能はやや劣ります	いいえ	Claude 3.7 Sonnet と併用できます
Gemini 2.0 Pro	Claude 3.5 Sonnet と比べて品質は同程度ですが、速度は遅めです	いいえ	Claude 3.5 Sonnet の代替品
Gemini 2.5 Flash	速度が速く 、マルチモーダル能力が優れています	いいえ	
Gemini 2.0 Flash	速度が速く 、品質は中程度です	いいえ	シンプルなタスクの第一候補👍

モデル名	説明	推論モデルかどうか	利用シーン
DeepSeek V3	速度が速く、品質が高いです	はい	文書検索、大規模データベースのクエリを行うタスクに適しています
DeepSeek R1 Distill Llama 70B	回答品質が高く、速度は中程度です（DeepSeek V3 より遅い）	はい	多段階の推論と背景知識を必要とするタスクに適しています
DeepSeek R1	回答速度はやや遅めですが、 中国語に対する理解力が非常に高く 、 回答内容の品質が高い です。深く思考し、役割指示の内容に確実に沿って調整します	はい	複雑な複数ターンの中国語対話が必要なシーンや、複雑な役割指示を処理する場合に適しています👍

オープンソースモデル

以下は主流のオープンソースモデルの比較表です。オープンソースモデルに必要なハードウェアは GPU の章を参照してください。

モデル名	説明	Agent 対応	利用シーン
GPT-OSS-120B	表形式で情報を整理して回答するのが得意	はい	データ分析、コピー制作
Gemma3 27B	画像 OCR の効果が良い	いいえ	請求書認識、画像分析
Meta Llama3.3 70B	コストパフォーマンスが極めて高い汎用モデルで、指示への忠実さが高い	いいえ	音声カスタマーサポート、RAG 質疑応答アシスタント
Meta Llama3.2 90B	視覚能力を備え、画像とテキストを同時に処理できる	いいえ	専門領域の質疑応答、高精度タスク
Meta Llama3.1 405B	知識の幅広さと推論能力が極めて高い	はい	カスタマーサポートの質疑応答、ナレッジ質疑応答
Llama3 Taiwan 70B	繁体字中国語と台湾文化に向けてファインチューニングされ、ローカライズされた用語が的確	いいえ	カスタマーサポートの質疑応答、RAG 質疑応答アシスタント
DeepSeek R1	推論を強化したモデルで、思考の連鎖を通じて複雑な問題を解決するのが得意	いいえ	科学的な論理推論
DeepSeek V3.2	MoE アーキテクチャで、推論速度が速くコストが極めて低い	いいえ	大量テキストの要約

モデル名	説明	Agent 対応	利用シーン
Qwen3 235B	プログラムコードと数学に特化し、極めて深い論理理解力を持ち、ハードコアなタスクに適している	はい	学术论文の補助
Qwen3 32B	高品質な回答と高効率な処理の間で動的にバランスを取る	はい	多言語対話と指示への忠実さ
Qwen3 8B	品質は中程度で、性能とコストのバランスが取れている	はい	チャットアシスタント、カスタマーサポートのよくある質疑応答
Qwen2.5 VL 72B instruct	視覚言語モデルで、画像の細部を的確に識別できる	はい	マルチモーダルチャットアシスタント
Mistral Large (24.07)	品質は中程度で、深い推論能力には欠けるが、速度が速い	いいえ	カスタマーサポートの質疑応答、シンプルなテキスト生成

モデルは必ず Fine-tune が必要ですか？

人工知能技術の急速な発展に伴い、言語モデルはすでに強力な言語理解および生成能力を備え、多くの領域で広く応用されています。例えば、事前学習済みの言語モデルは、日常会話、文章生成、シンプルな質疑応答タスクに容易に対応できます。

しかし、医療、法律、技術サポートなど、より挑戦的な専門領域のタスクにモデルが直面する場合、事前学習済みモデルだけに頼っていては最適な性能を提供できないことがあります。一部の開発者は **Fine-tuning**、すなわち特定の領域に向けて追加の訓練を行い、モデルの専門知識を高めることを選択します。

しかし、Fine-tuning は唯一の解決策ではなく、この目標を達成するための有効な方法は他にも 2 つあります。 **Prompt Engineering** と **RAG (Retrieval-Augmented Generation)** です。

1. Prompt Engineering：精確なプロンプトによるモデル性能の最適化

Prompt Engineering とは、精確なプロンプト文を設計してモデルが求める結果を生成するよう導くことです。この方法の核心は、タスクのニーズに応じて詳細な表現方法を設計し、モデルが回答の範囲を絞り込み、文脈と求められる出力形式を理解できるよう助けることにあります。

ある言語モデルがあり、ユーザーのニーズに応じて適切な製品を推薦することを目的としているとします。この場合、「高性能なスマートフォンを買いたい」と表現すると、モデルが十分に精確でない回答を生成してしまう可能性があります。なぜなら「高性能」という言葉には、処理速度、カメラ性能、バッテリー寿命など、さまざまな解釈があり得るからです。

推薦の精度を高めるために、**Prompt Engineering** を行い、より具体的な質問を設計したり、追加の文脈を提供したりして、ユーザーのニーズをモデルがよりの確に理解できるよう導くことができます。例えば、以下のようなプロンプト文に変更します。

「**長時間のバッテリー駆動と高効率なプロセッサ**を備え、価格が **500 ドルから 800 ドル**の間のスマートフォンが必要です。条件に合うスマートフォンをいくつか推薦してください。」

2. RAG：外部知識を組み合わせて生成能力を向上

RAG は、外部知識の検索と生成プロセスを組み合わせることで、モデルの性能を向上させる手法です。

従来の生成タスクでは、モデルは事前学習時に学んだ知識のみに頼っていました。一方 RAG は検索システムを活用し、外部資料をリアルタイムで取得して、それらの資料を生成モデルと組み合わせることで、より正確に質問に答えたりテキストを生成したりします。

例を挙げると、医療領域においてモデルがある稀少疾患について質問された場合、RAG はまず専門の医学データベースから関連資料を検索し、それらの資料に基づいてより正確な回答を生成できます。この方法の利点は、モデル自体が訓練過程で見たことのない資料であっても、既存の知識を検索することで高品質な応答を行える点にあります。特に、知識をリアルタイムで更新する必要があるシーンに適しており、モデルの知識範囲を大幅に拡張できます。

RAG のより詳しい紹介については、次章の「RAG ナレッジベース検索システムの説明」で行います。

まとめると、Fine-tuning、Prompt Engineering、RAG にはそれぞれ長所と適用範囲があり、単一の Fine-tuning 手法だけに頼るのではなく、応用シーンやニーズに応じて最も適した戦略を選択できます。

Prompt Engineering は、低コストで柔軟な解決策を提供し、精確なプロンプト文を設計することでモデルが高品質な結果を生成するよう導きます。

RAG は、外部知識と生成能力を組み合わせる手法を提供し、知識を動的に取得する必要がある場合に、より精確な回答を提供できます。

Fine-tuning はモデルの特定領域での性能を大幅に向上させることができますが、大量の専門資料を必要とし、追加のリソースを消費します。**Prompt Engineering** と **RAG** の両方でうまくいかなかった場合に実行する最後の手段と考えることができます。

RAG ナレッジベース検索システム

外部データベースまたはナレッジベースから情報を検索し、その検索結果を大規模言語モデルと組み合わせることで、より正確で文脈に即した回答を生成します。

RAG 技術の中核は、生成 AI の言語能力とナレッジ検索能力を組み合わせることです。モデルは質問に回答する際、内部の学習データだけに依存せず、外部データベースから最新かつ専門性の高い情報を動的に取得し、その情報を生成する回答に取り込むことができます。

RAG の処理フロー

高精度な RAG システム

RAG ナレッジベース検索システムは、ベクトル検索（Vector Search）を使用することで基本版を迅速に構築できますが、回答精度をさらに向上させることは容易ではありません。回答精度はユーザー体験にとって極めて重要であり、システムの回答に対するユーザーの信頼度と満足度に直接影響します。回答精度が不十分な場合、ユーザーはシステムの回答に疑問を抱き、利用意欲が低下する可能性があります。

2023 年の OpenAI 開発者会議で公開された情報によると、RAG システムは単純なベクトル類似度検索（Vector Search）のみを使用し、適切な埋め込みモデル（Embedding Model）を選択した場合、45% の精度を達成できます。さらに、HyDE Retrieval、FT Embeddings、Chunk/Embedding Experiments を追加すると、回答精度を 65% まで高めることができます。

MaiAgent RAG は、OpenAI 開発者会議で紹介された RAG 技術に加え、さまざまな従来型 NLP アルゴリズムと独自の検索技術を組み合わせています。社内データセットを用いて OpenAI RAG と回答の正確性を比較した結果、どちらも 95% の回答精度を達成しています。

MaiAgent RAG の回答精度

MaiAgent プラットフォームでは、MaiAgent RAG と OpenAI RAG の 2 種類の RAG を提供しています。各項目の比較は次のとおりです。

	MaiAgent RAG	OpenAI RAG
モデル対応	すべてのモデルに対応 🍌	OpenAI モデルのみに対応
環境対応	クラウドとオンプレミスに対応 🍌	クラウドのみに対応し、データを OpenAI に送信する必要があります

	MaiAgent RAG	OpenAI RAG
回答精度	非常に高い 🍑	非常に高い 🍑
対応ファイル形式	一般的なすべての形式に対応 🍑 doc, docx, xls, xlsx, csv, ppt, pptx, pdf, txt, json, jsonl, md	xlsx、csv は非対応 jsonl は非対応 旧形式の Office ファイル（doc、xlsx、ppt）は非対応
文書内の画像	対応（現在は実験的な機能） 🍑	非対応
文書内の表	対応（現在は実験的な機能） 🍑	非対応
会話への添付ファイルのアップロード	対応 🍑	対応 🍑
データ分割の透明性	可視化 🍑	ブラックボックス
デバッグの難易度	標準 🍑	ブラックボックスのためデバッグ不可
Top K の調整	エンタープライズ版のカスタマイズ機能 🍑	非対応
Embedding モデルの切り替え	エンタープライズ版のカスタマイズ機能 🍑	非対応

Embedding モデル

Embedding とは？

RAG（Retrieval-Augmented Generation）の処理において、**Embedding** の中心的な役割は、Parser で処理された文書やナレッジベースのコンテンツなど、大量のテキストデータを、コンピューターが理解して比較できる数値形式、すなわちベクトル（vectors）に変換することです。

この変換により、RAG システムは次のことが可能になります。

1. 意味を理解する

Embedding モデルは、表面的な語彙だけでなく、テキストの深い意味を捉えます。そのため、クエリと文書で使用される表現が異なっても、意味が近ければ、システムはそれらの関連性を認識できます。

2. 効率的に検索する

テキストをベクトルに変換した後、システムは効率的なベクトル類似度検索アルゴリズムを使用して、大規模なデータベースからユーザーのクエリに最も関連するテキスト部分を迅速に見つけられます。

3. 生成品質を向上させる

検索された関連テキストは、コンテキスト情報として大規模言語モデル（LLM）に提供され、LLM がより正確な回答を生成するのに役立ちます。

MaiAgent の RAG 技術における Embedding の2つの重要な役割

1. ナレッジベースのコンテンツをベクトルに変換し、ベクトルデータベースに保存します

2. ユーザーの質問をベクトル化し、データベース内のベクトルと類似度を比較して、最も関連するコンテンツを特定します

RAG の処理フロー

簡単に言えば、Embedding は RAG システムの基盤です。構造化されていないテキストデータを計算・比較可能なベクトルに変換するため、高精度な情報検索と高品質なコンテンツ生成を実現するうえで不可欠な前

処理です。

Embedding が RAG システムに与える影響

1. 検索品質

意味理解の深さ：高品質な Embedding モデルはテキストの意味をより正確に捉え、検索の関連性を高めます

コンテキスト認識：優れた Embedding モデルはテキストのコンテキスト関係を理解し、一貫性のある検索結果を実現します

多言語対応：強力な多言語 Embedding モデルは、言語をまたぐナレッジ検索のニーズに対応できます

2. システム性能

検索速度：Embedding モデルのベクトル次元数と計算効率は、検索の応答時間に直接影響します

リソース消費：Embedding モデルによって必要なコンピューティングリソースが異なり、システムの拡張性に影響します

並列処理：効率的な Embedding モデルは、大規模な並列検索に対応できます

MaiAgent が対応する Embedding モデル

モデル名	モデル開発元	開発国	特徴	オープンソース	デプロイ方式	MTEB 平均スコア
Cohere Embed v4.0	Cohere	カナダ	多言語対応、最高レベルの性能	いいえ	クラウド API 推論サービスの利用が必要	未公開。 v3.0 の 64.47 を参考にできます
Cohere Embed Multilingual v3.0 (Bedrock)	Cohere	カナダ	多言語対応、高性能	いいえ	クラウド API 推論サービスの利用が必要	64.47
OpenAI text-embedding-3-Large	OpenAI	米国	多言語対応（特に英語のコンテキストで高性能）、中程度の性能	いいえ	クラウド API 推論サービスの利用が必要	64.68

モデル名	モデル開発元	開発国	特徴	オープンソース	デプロイ方式	MTEB 平均スコア
EmbeddingGemma	Google	米国	オープンソース、多言語対応、軽量	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	61.15
Mxbai-embed-large	Mixedbread AI	米国	オープンソース、性能とリソースのバランスに優れ、長いコンテキストで高性能	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	64.68
BGE-Large	BAAI	中国	オープンソース、多言語対応、軽量	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	64.23
Nomic-embed-text	Nomic AI	米国	オープンソース、軽量	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	62.39
Qwen3-Embedding 0.6B	Alibaba	中国	オープンソース、軽量	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	61.82
Granite-embedding-278m-multilingual	IBM	米国	オープンソース、多言語対応、軽量	はい	ニーズに応じてクラウドまたはオンプレミス GPU にデプロイ可能	56.1

意味表現の精度と検索の正確性を確保するため、MaiAgent が提供する Embedding モデルは、MTEB (Massive Text Embedding Benchmark) 標準に基づいて選定・評価されます。MTEB は現在広く使用されている意味ベクトル化モデルの比較ベンチマークで、次のような多様なタスクを対象としています。Retrieval (意味検索) Classification (分類) Clustering (クラスタリング) Reranking (再ランキング) STS (Semantic Textual Similarity: 意味的テキスト類似度) Summarization QA Pair Classification など

MaiAgent の Embedding 技術の強み

1. 柔軟なモデル選択：

さまざまなニーズに対応する複数の Embedding モデルを提供します

2. 多様なデプロイ方式：

クラウドとオンプレミスのデプロイに対応し、データセキュリティを確保します

3. パフォーマンス最適化：

RAG のユースケースに合わせて特別に最適化し、最適な検索結果を提供します

4. コスト効率：

実際のニーズに応じて適切なモデルを選択し、性能とコストのバランスを取ります

Reranker モデル

Reranker とは？

Reranker は、**ランキングタスク**に使用されるモデルまたはシステムであり、通常は情報検索、レコメンデーションシステム、自然言語処理（NLP）で使用されます。主な目的は、候補結果の集合を並べ替え、最も関連性が高い、または最も適した選択肢を返すことです。

Reranker の仕組み

通常、検索エンジンやレコメンデーションシステムが検索結果、おすすめ商品、記事などの候補を返す際、それらはすでに何らかの初期ランキング基準に基づいて並べられています。この初期ランキングは、単純な一致や基本的な関連性指標に基づく場合があります。しかし、このようなランキングでは、すべての詳細やより深い意味的関係を十分に考慮できず、ユーザーのニーズに完全には合わない結果が生じる可能性があります。

たとえば、インターネットで「一番おいしいピザ店」を検索するとします。検索エンジンは、店名、評価、住所などの基本的な基準に基づいて多数の結果を返します。これらの結果はキーワードや単純な一致によってすでに並べられている場合がありますが、必ずしもニーズを完全に満たすとは限りません。

そこで Reranker が、さらに「**ランキングを最適化**」します。過去の検索傾向、ほかのユーザーによる店舗の評価、店舗の最新情報など、より多くの詳細を考慮して結果を確認し、並べ替えます。そのうえで最も関連性の高い店舗を選び、最適な選択肢をすばやく見つけられるようにします。この例における Reranker の処理フローは次のとおりです。

初期ランキング：単純なランキングモデル（キーワード一致、TF-IDF、その他の特徴量など）を使用して候補を最初に並べ替えます

「一番おいしいピザ店」を検索すると、検索エンジンが店舗の一覧を返します

候補結果の絞り込み：文脈理解、意味的な関連性、ユーザー行動など、より多くの特徴量に基づいてランキングを改善し、候補を再評価します

店舗をさらに詳しく確認し、よく訪れる地域にあるか、ほかのユーザーから高く評価されているか、キャンペーンを実施しているかなどを考慮します

最終ランキング：評価結果に基づいて、最も関連性が高い、または最も適した結果を返します

Reranker は、ニーズに最も合う店舗を上位に配置し、最適な選択肢を見つけやすくします

Cohere Rerank v3.5 の特長

Cohere Rerank v3.5 は、Cohere が提供する **再ランキング（reranking）** モデルです。検索結果や回答候補のランキングを最適化するために設計されており、候補結果を精密に並べ替えます。より深い意味理解と文脈に基づいて最も関連性の高い結果を選択し、最終的な品質を向上させます。

Parser 解析ツール

RAG Parser とは？

RAG Parser は Retrieval-Augmented Generation (RAG) システムの重要な工程です。元データを解析・分割し、Embedding によるベクトル化の前処理を行います。後続のベクトル化と意味検索の基盤となるため、データ品質と検索結果に大きく影響します。

RAG の処理フロー

1. ドキュメント解析用 Parser

MaiAgent は、PDF、Word、Excel、画像などの形式に対応する 4 種類のドキュメント Parser を提供しています。

機能・特性	MaiAgent Parser (デフォルト)	MaiAgent Parser (Online)	MaiAgent Parser (Offline)	Vision Parser
コスト	低	高	中	高
画像内容の解析	画像内の文字は解析できません	OCR で画像内の文字のみを解析します	OCR + AI で画像の意味を理解します	AI による視覚的理解で、最も高い効果が得られます
LLM の使用	いいえ	はい	はい	はい
テキスト解析の品質	標準	良好	良好（構造の保持に優れます）	良好
表の解析	ネイティブ抽出	AI で画像内の文字を抽出します	元の構造を保持します（最良）。視覚的理解に加えて画像の静的リソースも含みます	AI の画像認識により、画像を説明するテキストを生成します
解析時間	速い	中程度	遅い	遅い
オンプレミス（オフライン）	可能	不可	可能（VLM の導入が必要）	不可
対応形式数	22 種類	20 種類	20 種類（OCR 対応は PDF のみ）	7 種類

「自動検出（デフォルト）」と解析料金

PDF、Word（.docx）、PowerPoint（.pptx）をナレッジベースにアップロードする際、アップロード画面で各ファイルの横にある Parser のドロップダウンは、**自動検出（デフォルト）** に設定されています。MaiAgent Parser を固定で使用する設定ではありません。アップロード後、システムがファイルの構造を確認して Parser を選択します。

テキストのみで、目立った画像内容がないファイル → そのファイル形式のデフォルト Parser（通常は MaiAgent Parser）を使用します。

スキャン文書（テキストレイヤーなし）、画像内容を含むファイル（図面やスキャン画像など）、PowerPoint → OCR または視覚的理解に対応する Parser（MaiAgent Parser (Online)、MaiAgent Parser (Offline)、Vision Parser）から選択します。適切な Parser が見つからない場合は、デフォルト Parser を使用します。

判断基準は画像の枚数ではなく、ファイルの構造（テキストレイヤーの有無、画像や表を含むページ数）です。Word ファイルは、画像が 1 枚でも埋め込まれていれば画像内容を含むとみなされます。そのため、画像が少ない文書でも上記の Parser に切り替わる場合があります。実際に使用された Parser は、ナレッジベースのドキュメント一覧にある **Parser** 列で確認してください。クレジットは、**実際に解析を完了した Parser** に基づいて計算されます。

Parser	課金方法	取引履歴の表示
MaiAgent Parser	解析後のチャンク数に応じて課金します	Document parsing (maiagent): ファイル名
MaiAgent Parser (Online)	実際の課金対象ページ数に応じて、解析のたびに課金します	Document parsing (llama): ファイル名
MaiAgent Parser (Offline)	解析後のチャンク数に応じて課金します	Document parsing (docling): ファイル名
Vision Parser	その Parser に設定されたモデルの入力・出力 token に応じて課金します	Vision parsing input: ファイル名、Vision parsing output: ファイル名

ファイルの処理が完了した場合にのみクレジットを差し引きます。処理に失敗した場合は課金されません。同じファイルについて、Parser の種類ごとの課金はそれぞれ最大 1 回です（すべての Vision Parser は同じ 1 種類として扱います）。ドキュメント一覧で、そのファイルにまだ使用していない Parser に変更すると再解析され、新しい Parser に応じて課金されます。以前使用した Parser に戻す、同じ Parser で再解析する、または異なる Vision Parser 間で切り替える場合は、再度クレジットを差し引けません。例外として、MaiAgent Parser (Online) は実際の課金対象ページ数に基づき、解析のたびに課金します。この Parser に戻した場合や同じ Parser で再解析した場合も対象です。解析サービスのキャッシュが利用され、実際に課金されなかった場合はクレジットを差し引けません。過去の解析で差し引かれたクレジットは返還されません。そのため、同じファイルの取引履歴に複数の解析項目が表示されることがあります。ドキュメント一覧で Parser を変更すると（**Parser の一括変更**を含む）、確認画面を表示せずにすぐ再解析されます。解析コストを一定に保ち、別の Parser への自動切り替えを避けるには、アップロード時に MaiAgent Parser（または指定の Parser）を選択し、**自動検出（デフォルト）** を使用しないでください。

2. 音声テキスト変換用 Parser

MaiAgent は、音声ファイルを文字起こししてナレッジベースに取り込むための 4 種類の Parser を提供しています。

機能・特性	Azure Speech	Whisper (Groq)	Whisper (OpenAI)	Whisper (Offline)
コスト	高	低	中	無料
文字起こしの精度	高	高 (large-v3)	高 (whisper-1)	高 (ローカルモデルによる)
LLM の使用	いいえ	いいえ	いいえ	いいえ
解析速度	リアルタイム	最速	中程度	ハードウェアによる
オンプレミス (オフライン)	不可	不可	不可	可能
多言語対応	はい	はい (自動検出)	はい (自動検出)	はい (自動検出)
カスタムプロンプト	いいえ	はい	はい	はい
VAD 音声検出	—	はい	はい	はい
データのプライバシー	クラウド (Azure)	クラウド (Groq)	クラウド (OpenAI)	完全にローカル

音声ファイルをナレッジベースにアップロードすると、システムが自動的に文字起こしを行い、トランスクリプトの表示とダウンロードができるようになります。

画像認識のサポート

VLM (Vision Language Model) とは？

視覚言語モデル（Vision Language Model, VLM）は、画像の内容を理解し、画像情報とテキスト情報を組み合わせることができる先進的な人工知能モデルです。従来の画像処理技術とは異なり、VLM は単に画像を「見る」だけでなく、画像内の物体・シーン・関係性を「理解」し、画像の内容に基づいてテキストを生成したり、質問に回答したり、関連する指示を実行したりすることができます。

VLM の中核となる能力は、そのマルチモーダル処理能力、すなわち異なるソース（視覚とテキスト）からの情報を同時に処理・理解する能力にあります。これにより、VLM はより複雑で、より文脈を認識したタスクを実行できます。

VLM と従来の OCR の比較

従来の光学文字認識（OCR）技術は、画像から文字を検出して抽出することに特化していますが、画像全体の意味や非文字コンテンツを理解する面では限界があります。

特性	従来の OCR	VLM (Vision Language Model)
主な機能	画像から文字を抽出する	画像の内容を理解し、テキスト情報と組み合わせる
情報処理	単一モーダル（文字ピクセルのみを処理）	マルチモーダル（画像の視覚情報とテキスト意味を同時に処理）
理解の層	文字レベルの認識	シーン理解、物体識別、関係推論、文脈認識
主なタスク	文書スキャン、文字抽出	画像説明文の生成、視覚的質問応答 (VQA)、画像検索、物体検出など
文脈認識	限定的。主に後処理の言語モデルに依存する	強力。画像全体の状況と細部を理解できる
非文字コンテンツの処理	通常は無視するか処理できない	画像内の物体・シーン・動作などを識別し理解できる

VLM の利点は次のとおりです。

- より深い理解**：VLM は文字を読み取るだけでなく、画像の意味的な内容も理解できます。たとえば、物体の識別、シーンの分析、画像内の要素間の関係性の理解などです。
- インタラクティブ性**：VLM は視覚的質問応答（Visual Question Answering, VQA）を行うことができ、画像の内容に基づいてユーザーの質問に回答します。

- * **コンテンツ生成**：VLM は画像に対して説明文（Image Captioning）を生成できます。
- * **多機能性**：テキスト関連のタスクに加えて、VLM はより幅広い視覚理解タスクにも応用できます。

VLM の実際の活用事例

VLM の強力な機能により、さまざまな分野で幅広い応用が見込まれています。

視覚的質問応答 (VQA)

応用：ユーザーが画像をアップロードし、その内容について質問します。たとえば「画像の中の人物は何色の服を着ていますか？」や「この写真は室内と屋外のどちらで撮影されましたか？」などです。

シーン：スマートアシスタント、教育、視覚障がい者支援。

画像説明文の生成 (Image Captioning)

応用：画像に対して簡潔で正確な説明文を自動生成します。

シーン：画像の自動タグ付け、コンテンツ管理システム、ソーシャルメディア向けコンテンツ生成、視覚障がい者の画像理解支援。

コンテンツベースの画像検索

応用：自然言語による説明を用いて画像を検索できます。たとえば「オフィスで人が会議をしている画像を探す」などです。

シーン：大規模な画像ライブラリの管理、Eコマースの商品検索。

マルチモーダルデータ分析

応用：医用画像とカルテのテキストを組み合わせることで医師の診断を支援する、商品画像とユーザーレビューを分析して市場トレンドを予測する、などです。

シーン：医療、小売、金融。

人間とのインタラクション

応用：ロボットや仮想アシスタントが視覚的な環境を理解し、それに基づいて人間とより自然にインタラクションできるようにします。

シーン：スマートロボット、自動運転車（交通標識や路面状況の理解）。

MaiAgent 画像認識の優位性

MaiAgent RAG は VLM 技術を統合し、技術者や開発者に対して便利で効率的な画像認識・理解ソリューションを提供します。以下の優位性を備えています。

添付画像の VLM 認識（画像理解と質問応答）

MaiAgent RAG は、ユーザーがアップロードした添付画像に対して VLM 認識を行い、画像内容を精密に理解します。

添付文書に埋め込まれた画像の VLM 認識

MaiAgent RAG は、添付文書（PDF、Word 文書など）内部に埋め込まれた画像の VLM 認識機能を積極的に開発しています。これにより、システムは文書内の文字を理解するだけでなく、画像の内容も解析でき、真の意味でのマルチモーダル文書理解を実現します。これは多くの標準的な RAG システムには備わっていない高度な機能です。

添付文書内の画像コンテンツに対する質問応答

添付画像であっても、文書に埋め込まれた画像であっても、MaiAgent RAG は画像内容に基づく質問応答をサポートします。ユーザーは画像の細部について直接質問でき、精密な回答を得られます。

ナレッジベース文書内の画像コンテンツに対する質問応答

MaiAgent RAG はナレッジベース文書内の画像に対する質問応答をサポートしています（ロール指示の Prompt で、画像を Markdown 形式で表示するよう指定する必要があります）。

ナレッジベース文書内の画像を VLM 認識した後の質問応答

MaiAgent は、ナレッジベース文書内の画像をまず VLM 認識し、その認識結果と組み合わせて深い質問応答を行う実験版機能を提供します。これにより、画像情報とナレッジベースの連携がさらに進み、より包括的なナレッジサービスを提供します。

MaiAgent の VLM 技術を活用することで、画像情報の価値をより深く掘り起こし、よりスマートな人間とのインタラクションと自動化フローを実現できます。

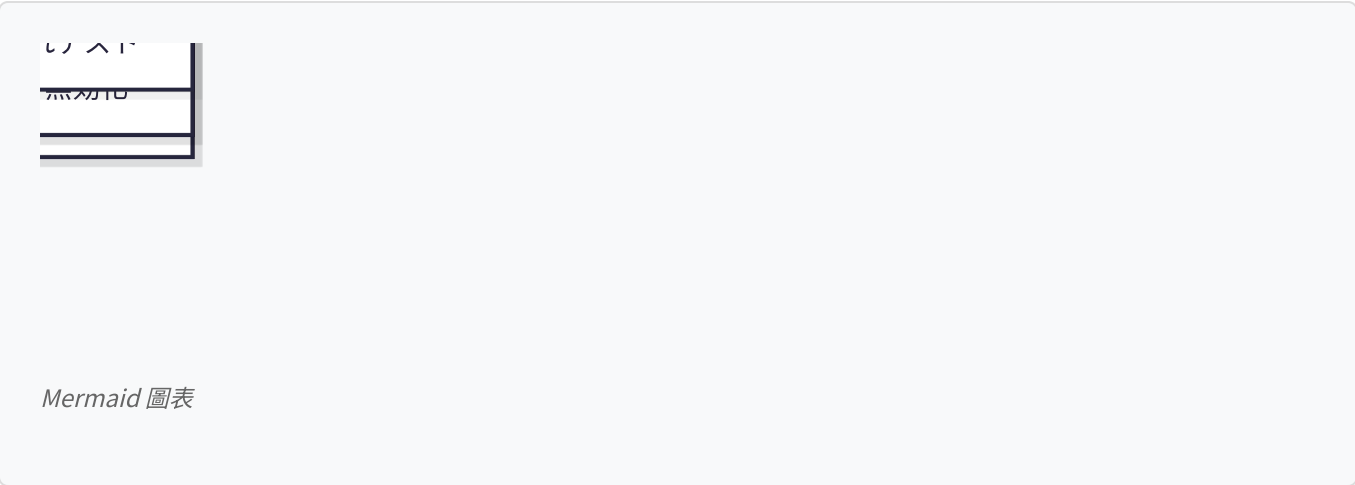
MaiAgent Work | 従業員の生産性向上のための連携

自社システムへの MaiGPT の追加

このページは、「自社システムにMaiGPTを追加する」エンジニアが最初から最後まで通して読めるように構成されています。企業のポータルサイト、社内ワークスペース、顧客向けに運用するSaaSのいずれであっても、ユーザーが **MaiAgent組織のメンバー**（従業員、担当者）であれば、手順は同じです。各ステップには実施事項を直接記載し、詳しく知りたい場合にのみリンクを開けるようにしています。

ユーザーがお客様、会員、匿名の訪問者であり、組織メンバーではない場合、MaiGPT製品自体を利用することはできません。代わりに[独自のAI Agent+MaiGPTウィンドウモード](#)を使用してください。連絡先IDの連携については、[ソフトウェアベンダー向け連携ガイド](#)を参照してください。

全体像



完了すると、従業員が自社システムにログインした際、ページ内にMaiGPTが直接表示されます。画像生成、Deep Research、コネクタを利用でき、権限、Credits、会話履歴はすべてMaiAgentのメンバーIDに紐づきます。従業員がMaiAgentに別途ログインしたり、MaiAgentの存在を意識したりする必要はありません。

開始前の準備

項目	取得場所
MaiGPTとID同期を含むプランのMaiAgent組織	不明な場合は営業担当者にお問い合わせください
組織オーナー、または「組織概要」の管理権限を持つユーザーが所有する組織API Key	取得方法は クイックスタート を参照してください

項目	取得場所
MaiGPTのWeb Chat ID	<code>GET /api/v1/web-chats/</code> のレスポンスから、名前が <code>MaiGPT</code> で <code>accessType</code> が <code>web</code> の項目を選びます
ロールを事前に作成し、権限を設定しておく	バックエンドの「組織設定 → 組織概要 → ロール権限を編集」で、たとえば「従業員」ロールを作成し、MaiGPTの利用権限を有効にします（ 設定方法 ）。 メンバーのすべての機能はロールによって決まります 。この手順を行わないと、以降の設定はすべて機能しません
メンバーの登録元を決める	次のセクションにある3つの方法から選びます

ステップ1：メンバーをプロビジョニングする

ユーザーは、まずMaiAgent組織のメンバーである必要があります。次の3つから最も簡単な方法を選んでください。

組織内ですでにMaiAgentを利用しているユーザーがいても、SCIMを接続することで既存のアカウントが無効になったり、重複アカウントが作成されたりすることはありません。管理対象となった既存メンバーは、その後、自社システムと同期されます。詳しくは[既存のアカウントとメンバーへの影響](#)を参照してください。

自社システムにユーザー管理がある場合（推奨）

自社システムでユーザーを追加、変更、無効化するイベントからMaiAgentのSCIMエンドポイントを呼び出し、メンバーをリアルタイムで同期します。

バックエンドの「組織設定 → ID同期」でSCIMを選択し、「接続情報」タブからEndpoint URLとTokenをコピーします。Tokenは一度しか表示されないため、サーバーの環境変数に保存してください。

[リアルタイム同期ページ](#)の `MaiAgentMembers` clientをプロジェクトに追加し、イベント内で呼び出します。

```
members = MaiAgentMembers()    # 環境変数 MAIAGENT_SCIM_URL MAIAGENT_SCIM_TOKEN を
読み取ります

# ユーザーを追加します
user.maiagent_member_id = members.on_user_created(
    external_id=user.id, email=user.email, name=user.display_name, groups=["従業員"],
)
# 名前または所属部署を変更します
members.on_user_updated(user.maiagent_member_id, user.id, user.email,
user.display_name, ["従業員"])
# アクセスを停止、または退職処理を行います
members.on_user_deactivated(user.maiagent_member_id)
```

初回リリース前に、既存ユーザー全員に対して `on_user_created` を一度実行します（幕等です。一括バックアップを参照してください）。以降はイベントによって同期状態を維持します。

名簿のみがある場合（人事システムからのエクスポート、Excel、データベース）

名簿同期ツールを使用します。社員番号、Email、氏名、ロールの4列を持つCSVまたはJSONを用意し、SCIM接続用の環境変数を設定します。最初に `--dry-run` で計画を確認してから実行し、その後は1日1回の定期実行を設定します。名簿にいるユーザーは作成または更新され、名簿からなくなったユーザーは無効化されます。

Okta／Entra ID／JumpCloudを使用している場合

SCIMのEndpoint URLとTokenをIdPのプロビジョニング設定に入力します。IdPからユーザーとグループが自動的にプッシュされます。詳しくはIdPからのプッシュを参照してください。

実際の連携で特につまづきやすい、次の3点に注意してください。

ロール名をバックエンドで作成したロールに一致させます：SCIMでメンバーを作成するときに `groups` へ指定する名前は、バックエンドで権限を設定したロール名と一致させてください。一致しない場合、**権限のない新しいロール**が自動的に作成されます。

Emailがアカウントのキーです： `userName` は、後でtokenの発行に使用するEmailです。作成後はSCIM経由でEmailを変更できません。

SCIMで作成したアカウントにはパスワードがありません：この方法を利用する従業員はMaiAgentにログインしません。これは意図した動作です。MaiGPTのWeb版またはAppも利用可能にする場合は、別途SSOを設定してください。

ステップ2：バックエンドでメンバーtokenを発行する

バックエンドに、**ログイン済みユーザー**のみが呼び出せるエンドポイント（例： `GET /maigpt-token` ）を作成します。内部では、組織API Keyを使って、そのメンバー用の短期間有効なtokenをMaiAgentから発行します。

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <組織API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com"}'
```

成功すると `201` が返されます。

```
{
  "accessToken": "<メンバー用の短期間有効なJWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": { "id": "c1a4 ... ", "userId": "5e77 ... ", "email": "alice@acme.com",
  "name": "Alice" }
}
```

レスポンス全体を変更せずにフロントエンドへ返してください。ルールとステータスコードは次のとおりです。

仕様	説明
有効期間は15分で、refresh tokenは発行されません	更新はフロントエンドSDKが自動的に処理します（ステップ3）。バックエンドは「呼び出されるたびに1つ発行する」だけです。 毎回新しいtokenを発行し、キャッシュしないでください
tokenはMaiGPTの会話エンドポイントでのみ使用できます	<code>/api/v1/</code> 配下のAPIで使用すると、常に <code>401</code> が返されます。漏えいしても組織データにはアクセスできません
<code>401</code> / <code>403</code>	Api-Keyを使用していない、またはKeyの所有者がオーナーでも組織管理者でもありません
<code>404</code>	このEmailは、この組織の有効なメンバーではありません。ステップ1を実施していないか、メンバーが無効化されています
発行のたびに監査ログが記録されます	組織のAPIリクエストログで確認できます。レスポンス内のtokenは記録されません

パラメータと詳細については、[メンバーtokenの発行](#)を参照してください。

ステップ3：フロントエンドに埋め込む

MaiGPTを表示するページに埋め込みコードを追加し、`auth.getToken` をステップ2のエンドポイントに向けます。それ以外はSDKが処理します。初期化時にtokenを一度取得し、有効期限が切れる前に自動的に再取得し、同じ会話を継続して使用します。

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPTのWeb Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',           // 任意：左上に表示するブランド名
    locale: 'ja',                     // 任意：インターフェースの言語
    auth: {
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  },
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`sourceId`には、必ずレスポンス内の`member.userId`を指定してください。`member.id`や自社システムのユーザー番号ではありません。MaiGPTは、この値を使って会話をメンバーに紐づけます。誤った値を指定すると匿名の会話になり、そのメンバーの履歴を確認できなくなります。

コンテナの高さは自分で設定してください。SPAでは、コンテナがmountされてからSDKを読み込んでください。`baseUrl`とSDKのURLはSaaSのアドレスです。プライベート環境にデプロイする場合は、自社のドメインに置き換えてください。レイアウトとその他のパラメータについては、[MaiGPTモードの埋め込み](#)を参照してください。

ステップ4：ログアウトと退職処理

イベント	実施すること	結果
ユーザーが自社システムからログアウトする	フロントエンドで <code>MaiAgent.auth.signOut()</code> を呼び出します	iframeのID情報が消去され、更新が停止します。発行済みのtokenは最長15分間有効です
ユーザーが退職する／アクセスを停止する	ステップ1の方法で無効化します（ <code>on_user_deactivated</code> 、名簿からの削除、IdPでの無効化）	以降の発行では <code>404</code> が返され、埋め込みページは自然に利用できなくなります。会話とCreditsの記録は保持されます
復職する	ステップ1の方法で再有効化します	ロールと履歴が復元されます

ステップ5：受け入れテスト

順番に確認してください。各項目には明確な判定基準があります。

メンバーが存在し、ロールが割り当てられている：バックエンドの「メンバー管理」に対象者が表示され、設定した権限を持つロールが割り当てられていることを確認します。

発行に成功する：ステップ2のcurlで `201` が返されます。存在しないEmailでは `404` が返されます。

会話できる：埋め込みページでMaiGPTを開いてメッセージを送信し、応答が返ることを確認します。そのメンバーとしてMaiGPTのWeb版にログインする（SSOがある場合）か、バックエンドの会話記録でその会話を確認します。

機能が正しい：画像の生成を依頼します。ロールでコネクタとDeep Researchが有効になっている場合は、その入口が表示されます。

更新を意識せず利用できる：ページを15分以上開いたままにしてから、もう一度メッセージを送信します。ページを再読み込みせず、同じ会話を続けられることを確認します。

ログアウトが反映される： `signOut()` の後にメッセージを送信すると拒否されます。メンバーを無効化すると、ステップ2で `404` が返されます。

リリース後

SCIM TokenとAPI Keyのローテーション：SCIM Tokenの作成から180日を超えると、バックエンドに通知が表示されます。再生成すると古いTokenは直ちに無効になるため、同時にサーバーの環境変数も更新してください。

プランの容量：メンバー数は、組み込みAgentのシート数とメンバーごとのCredits上限に影響します。[メンバーのCredits上限管理](#)を参照してください。

監査：tokenを発行するたびに、組織のAPIリクエストログへ記録されます。SCIMの各同期は、バックエンドの「ID同期 → 同期履歴」に記録されます。

トラブルシューティング

現象	原因	対処方法
発行時に <code>404</code> が返される	Emailが、この組織の有効なメンバーではありません	ステップ1に戻り、メンバーが存在して無効化されておらず、Emailが同じであること（大文字と小文字は区別されません）を確認します
発行時に <code>403</code> が返される	API Keyの所有者がオーナーではなく、組織の管理権限也没有ありません	管理者が発行したKeyを使用します
チャットウィンドウに「会話の作成に失敗しました（400）MaiGPTを使用する権限がありません」と表示される	メンバーのロールにMaiGPTの利用権限がないか、 <code>auth</code> が渡されていません	バックエンドのロール権限でMaiGPTを有効にし、 <code>getToken</code> が <code>accessToken</code> と <code>organizationId</code> を返していることを確認します
会話が匿名になり、履歴を確認できない	<code>sourceId</code> が間違っています	<code>member.userId</code> を使用します
15分後にメッセージを送信できない	<code>getToken</code> が呼び出されていないか、失敗しています（ブラウザのconsoleで <code>[MaiAgent]</code> のエラーを確認します）	ログイン済みユーザーがエンドポイントを利用でき、レスポンス形式が正しいことを確認します
メンバーを作成したが何も利用できない	ロールが割り当てられていないか、ロールの権限が空です	バックエンドでロール権限を設定し、SCIMで同じ名前のロールを割り当てます
Emailを変更しても反映されない	SCIMではEmailを変更できません	バックエンドで変更するか、古いアカウントを無効化して新しいEmailで作成します

その他のSCIMエラーコードについては、[名簿同期ツールのエラー一覧](#)を参照してください。

関連資料

[メンバーのプロビジョニング：SCIM／招待／SSO](#) — SCIM APIの完全なリファレンスと3つの方法の比較

[リアルタイム同期：ユーザーイベントから呼び出す／名簿同期ツール](#)

[MaiGPTの連携：適切なアシスタントを選び、機能を有効にする](#) — MaiGPT製品と独自アシスタントの違い、およびメンバーIDを埋め込むための全パラメータ

[Web Chatの埋め込みとSDK／MaiGPTモードの埋め込み](#)

MaiGPT 連携：適切なアシスタントの選択と機能の補完

MaiAgentプラットフォームで「MaiGPT」は2つの異なるものを指します。連携前に区別しておくことで、多くの手戻りを避けられます。

	MaiGPT製品（組織組み込みアシスタント）	MaiGPTウィンドウモード（Web Chat SDKのレイアウト）
概要	各組織に自動で提供され、MaiAgentが一元管理する企業向けAIワークスペースです。AI画像生成、Deep Research、コネクタなどの機能を内蔵しています。 MaiGPTとは？ をご覧ください。	<code>enabledWindowModes: ['maigpt']</code> で有効になる、会話履歴サイドバー、検索、設定を備えた ChatGPT風の完全なインターフェース です。フロントエンドの表示形式にすぎません。
利用者	組織の メンバー がログイン後に利用できます（Web、iOS App、LINE、Microsoft Teams）。権限は ロール利用権限 で制御します。	匿名訪問者や 連絡先ID を含め、すべてのWeb Chatで利用できます。
自社システムへの埋め込み	組織メンバー向けに限り可能です 。会話作成時に、 <code>maigpt_access</code> 権限を持つメンバーIDが必要です。そのWeb Chat IDを直接埋め込むと、 <code>400 MaiGPTを使用する権限がありません</code> が返されます。メンバー向けに埋め込む場合は、 メンバーIDによる埋め込み を使用し、バックエンドでメンバートークンを発行します。	可能です 。本ページの第2節と第3節で説明します。
カスタマイズ	できません 。指示、モデル、ツール、スキルはプラットフォームで一元設定されます。組織側ではロール権限による 制限 のみ可能です。	可能です 。画面で動作するのは独自に作成したAI Agentであり、指示、モデル、ツール、ナレッジベースを自由に設定できます。

結論：用途は2つに分かれます。**自社の顧客や組織外のユーザー向けには、独自のAI Agentを作成して必要なツールを追加し、MaiGPTウィンドウモードで埋め込みます**（第2節、第3節）。**組織メンバー向けで、画像生成、Deep Research、コネクタ、ロール権限などMaiGPT製品自体の機能が必要な場合は、メンバーIDによる埋め込み**（第5節）を使用し、バックエンドから短期トークンを発行します。

1. よくある症状

表示される現象	原因	解決方法
画面はMaiGPTに見えますが、応答は通常のChatと同じで、画像生成を依頼するとAIが「画像を生成できません」と返します。	埋め込まれているのは通常のAI Agentで、 （1）回答モードが Agentモード ではない、 または（2）画像生成などのツールが追加されていません。	第2節 に従ってAgentモードに切り替え、ツールを追加します。

表示される現象	原因	解決方法
チャットウィンドウが開かず、「会話の作成に失敗しました（HTTP 400）」と表示され、APIから <code>MaiGPTを使用する権限がありません。</code> が返されます。	埋め込んだWeb Chat IDが組織組み込みのMaiGPT製品に属していますが、訪問者にはメンバーIDがありません。	顧客向け → 独自アシスタントのWeb Chat IDを使用します（ 第3節 ）。メンバー向け → メンバーIDによる埋め込み に従ってメンバートークンを渡します。
管理画面のAI Agent一覧APIにMaiGPTが表示されません。	<code>GET /api/v1/chatbots/</code> には組織が独自に作成したアシスタントのみが表示され、MaiGPT製品は含まれません。	これは想定された動作です。連携では必ず独自アシスタントを対象にします。



組織組み込みMaiGPTのWeb Chatを埋め込むと、チャットウィンドウに会話作成失敗が表示されます

組織組み込みMaiGPTのWeb Chatを埋め込んだ場合：訪問者はログイン済みメンバーではないため、会話作成が拒否されます（HTTP 400）

2. 画像を生成し、ツールを使用できるAI Agentを作成する

以下では「AI画像生成」を例にします。Web検索、Web取得、MCPツール、独自のAPIツールなど、その他の機能も同様に追加できます。

AI Agentを作成する

管理画面の「AI機能 → AIアシスタント」で新しいAI Agentを作成し、ロール指示とモデルを設定します。操作方法はユーザーマニュアルの[AIアシスタントの作成](#)をご覧ください。

回答モードを「Agentモード」に切り替える

アシスタントの「回答モード設定」を開き、回答モードを**Agentモード**に変更します。

ツールを呼び出せるのはAgentモードだけです。デフォルトの「RAG Q&A」モードは応答が速い一方、ツールを一切使用しません。画像生成ツールを追加したのに画像を生成しない場合の最も一般的な原因です。

ツールを追加する

アシスタントの「ツール」ページを開き、「ツールを選択」をクリックして、必要な組み込みツールを選択します。例：

画像生成： `Global_Image_Tool_OpenAI_GPT_Image_2`、`Global_Image_Tool_Gemini_3` など（いずれか1つを選択します。モデルの違いは[組み込みAI画像生成ツール](#)をご覧ください）

Web検索： `Global_Tavily_Search_Tool`

Web取得： `Global_Web_Fetch_Tool`

独自システムのAPIツールまたはMCPツール

詳しい操作方法は、ユーザーマニュアルの[AIアシスタントへのツール設定](#)をご覧ください。

Web Chat会話プラットフォームを作成する

「カスタマーサービス会話 → 会話プラットフォーム」で、このアシスタント用の**Web Chat**を追加し、`webChatId`を取得します。[会話プラットフォーム連携：Webサイト](#)をご覧ください。

APIで自動化する（ISVによる顧客環境の構築）

上記4つの手順には対応するAPIがあり、ソフトウェアベンダーは顧客ごとの開通時にプログラムで処理できます。認証方法とAPI Keyの取得方法は[クイックスタート](#)をご覧ください。

1. 組み込みツールを一覧表示し、画像生成ツールのidを取得します

GET /api/v1/tools/

2. AI Agentを作成します：回答モードをagentにし、ツールを追加します

POST /api/v1/chatbots/

```
{
  "name": "Acme GPT",
  "instructions": "あなたはAcme社のAIアシスタントです。日本語で回答してください。",
  "largeLanguageModel": "<LLM id>",
  "embeddingModel": "<embedding model id>",
  "rerankerModel": "<reranker model id>",
  "replyMode": "agent",
  "tools": ["<Global_Image_Tool_OpenAI_GPT_Image_2のid>"]
}
```

3. アシスタント用のWeb Chat会話プラットフォームを作成します（レスポンスのchannel.idがwebChatIdです）

POST /api/v1/inboxes/

```
{
  "name": "Acme GPT",
  "chatbot": "<chatbot id>",
  "channelType": "web",
  "channel": {
    "name": "Acme GPT",
    "theme": { "primaryColor": "#1890ff", " ... ": " ... " },
    "darkTheme": { "primaryColor": "#404040", " ... ": " ... " }
  }
}
```

既存アシスタントのモード変更やツール追加には `PATCH /api/v1/chatbots/{id}` を使用します。`tools` にはツールIDの完全な配列を渡します（配列全体が置き換えられます）。

`theme` と `darkTheme` は必須です。フィールド一覧は、先に `GET /api/v1/web-chats/{既存のwebChatId}` を呼び出し、既存設定をコピーして取得できます。

Web Chatは、必ず `POST /api/v1/inboxes/` で受信トレイと一緒に作成します。`POST /api/v1/web-chats/` を直接呼び出すと、受信トレイのない孤立したWeb Chatが作成され、埋め込み時に404が返されます。

3. MaiGPTウィンドウモードで埋め込む

第2節で取得した `webChatId` を使用し、MaiGPTモードでの埋め込みに記載された埋め込みコードをページに配置します。

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<第2節で作成したwebChatId>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

アシスタントがログインユーザーを識別し、その権限でAPIを呼び出せるようにするには、ソフトウェアベンダー連携ガイドに従って `contactId` または `auth` を追加します。

検証：機能追加前後で同じ埋め込みコードを使用する

以下の2枚の画像は、同じAI Agentと同じ埋め込みコードを使用しています。違いは「回答モード+ツール」の設定だけです。

追加前：RAG Q&Aモード、ツールなし



MaiGPTウィンドウモードで、ツール未設定のアシスタントが画像を生成できないと回答します

インターフェースはMaiGPTウィンドウモードですが、アシスタントは「画像を生成、作成、制作できません」と回答します。

追加後：Agentモード+画像生成ツール



MaiGPTウィンドウモードで、画像生成ツールを設定したアシスタントが画像を直接出力します

同じ埋め込みコードで、アシスタントが画像生成ツールを呼び出し、会話内に画像を直接表示します。

4. MaiGPT製品自体を使用すべき場合

要件が「自社製品に埋め込んで顧客に提供する」のではなく、「従業員向けにChatGPTレベルの社内アシスタントを用意する」ことであれば、MaiGPT製品を直接使用するのが最も簡単です。

従業員は組織メンバーとしてログインすれば利用できます。入口には[Web版](#)、[iOS App](#)、[LINE](#)、[Microsoft Teams](#)があります。

画像生成、Deep Research、コネクタ、スキルは組み込み済みです。管理者は[ロール利用権限](#)で各機能を利用できるユーザーを決めるだけです。

会社固有の知識を追加するには、該当するナレッジベースをメンバーに公開します。別のアシスタントを作成する必要はありません。

「自社システムに埋め込み、利用者が組織メンバーである」場合は、第5節のメンバーIDによる埋め込みを使用します。「組織外のユーザー（顧客、会員）向け」「独自のAPIツールが必要」「指示やモデルを変更したい」のいずれかに該当する場合は、第2節、第3節の独自アシスタント方式を使用します。

5. メンバーIDによる埋め込み：組織メンバー向けにMaiGPT製品を自社システムへ埋め込む

最初から最後までの手順は、[MaiGPTを自社システムに追加する](#)をご覧ください。本節では各パラメータを網羅的に説明します。

この方法は、自社システムのユーザーが**MaiAgent組織のメンバーでもある**場合（企業ポータル、従業員ワークスペース、ISVが管理する企業内システムなど）に適しています。別のアシスタントではなくMaiGPT製品自体を利用する場合、**バックエンド**が組織API Keyを使ってログイン中のメンバー用の短期トークンを発行し、フロントエンドはSDKの既存のメンバーID注入設定でMaiGPTのWeb Chatを埋め込みます。

```
sequenceDiagram
    participant U as メンバーのブラウザ
    participant P as 自社バックエンド
    participant M as MaiAgent API
    participant W as MaiGPT Web Chat
    U->>P: ログイン済みで、MaiGPTを含むページを開く
    U->>P: SDKがauth.getTokenを呼び出す
    P->>M: POST /api/v1/members/embed-tokens/ (Api-Key、メンバーのemailを指定)
    M-->>P: accessToken (15分)、organizationId、member.userId
    P-->>U: getTokenに返す
    U->>W: SDKがメンバーIDで会話を作成し、メッセージを送信
    Note over U,W: tokenの期限が近づくとiframeがSDKに通知し、SDKがgetTokenを再度呼び出して自動更新する
```

Mermaid 図表

前提条件

組織API Keyが必要です。所有者が組織オーナーであるか、「組織概要」の管理権限を持っている必要があります。メンバーを管理できるユーザーが発行したKeyだけが、メンバーの代理でトークンを発行できます。API Keyの取得方法は[クイックスタート](#)をご覧ください。

埋め込むユーザーが組織メンバーとして登録済みで、ロールにMaiGPT利用権限が付与されている必要があります（[MaiGPTロール利用権限](#)）。メンバーが存在しない場合、発行処理は404を返し、自動登録は行いません。SCIMプロビジョニングAPI、管理画面からの招待、SSOなどの登録方法は[メンバーのプロビジョニング](#)をご覧ください。ISV管理環境の標準的な方法は、SCIMでメンバーを登録し、本節の方法でトークンを発行することです。

MaiGPTのWeb Chat IDが必要です。 `GET /api/v1/web-chats/` のレスポンスから、名前が `MaiGPT` で `accessType` が `web` の項目を使用します。

ステップ1：バックエンドでメンバートークンを発行する

ログイン済みユーザーだけが呼び出せるバックエンドエンドポイント（例： `GET /maigpt-token` ）を作成します。内部でMaiAgentを呼び出し、フロントエンドSDKが必要なときにこのエンドポイントを呼び出します。

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <組織API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com"}'
```

`email` と `memberId` のいずれか一方を指定します（ `memberId` はMaiAgentのメンバーIDです）。成功すると `201` が返されます（ `expiresAt` はミリ秒単位のepochです）。

```
{
  "accessToken": "<メンバー短期JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": {
    "id": "c1a4 ... - ... ",
    "userId": "5e77 ... - ... ",
    "email": "alice@acme.com",
    "name": "Alice"
  }
}
```

ステータスコード	意味
400	<code>email</code> と <code>memberId</code> の両方を指定したか、どちらも指定していません。
401	Api-Keyで認証されていません（メンバーログイントークンと匿名リクエストは拒否されます）。
403	API Keyの所有者が組織オーナーではなく、組織管理権限も持っていません。
404	指定したメンバーが存在しない、無効化されている、またはこのKeyの組織に所属していません。

トークンのルール：有効期間は15分に固定され、refresh tokenは発行されず、延長もできません。同じメンバーについて繰り返し発行でき、以前のトークンも期限までは有効です。このトークンはWeb Chatの会話にのみ使用できます（会話作成、メッセージ送信、引き継ぎ後の会話継続の3つの呼び出し）。/api/v1/の各エンドポイント、その他の会話管理操作、WebSocketに使用すると、常に401が返されます。発行のたびに監査用として組織のAPIリクエスト履歴に記録されます（レスポンス内のトークンは記録されません）。

ステップ2：SDKにトークン取得関数を渡す

埋め込み設定は第3節と同じですが、`auth.getToken` を追加します。`webChatId` はMaiGPTのWeb Chat IDに置き換えます。

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPTのWeb Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    auth: {
      // SDK初期化時に1回呼び出し、その後tokenの期限が近づくたびに再度呼び出して自動更新します。タイマーは不要です
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' }) // 自社バックエンド。ステップ1を参照してください
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`getToken` が返すオブジェクトのフィールド：

フィールド	必須	説明
<code>accessToken</code>	✓	ステップ1のレスポンスに含まれる <code>accessToken</code> です。
<code>organizationId</code>	✓	ステップ1のレスポンスに含まれる <code>organizationId</code> です。
<code>sourceId</code>	推奨	ステップ1のレスポンスに含まれる <code>member.userId</code> です。 <code>auth.sourceId</code> に設定することもでき、両方に指定した場合は <code>auth.sourceId</code> が優先されます。

SDKが最初の会話を作成する前にメンバーIDの注入を待つよう、`auth` は `maiagentChatbotConfig` 内に記述する必要があります。SDKの読み込み後に `MaiAgent.auth.setup()` を呼び出すと、ID注入前に最初の会話が匿名で作成され、拒否される可能性があります。

`sourceId`には、レスポンスの`member.userId`を指定する必要があります。`member.id`や自社システムのユーザー番号ではありません。MaiGPTはこの値で会話をメンバーに関連付けます。誤った値を指定すると匿名の会話となり、そのメンバーのMaiGPT履歴に表示されません。

有効期限と更新の仕組み

トークンの有効期間は15分です。期限の60秒前にiframeがSDKへ通知し、SDKが `getToken` を再度呼び出して新しいトークンを取得・注入します。**ページやiframeを再読み込みせず、同じ会話を継続して使用できる**ため、ユーザーには更新を意識させません。

`getToken` を呼び出すたびにMaiAgentから新しいトークンを発行し、キャッシュした値を返さないでください。 SDKはトークン自体の残り有効期間に基づいて次の更新をスケジュールします。前回と同じトークンを返すと失敗と見なされ、以下の再試行処理に移ります。

ユーザーがログアウトした場合やバックエンドが401を返した場合など、`getToken` が失敗すると、SDKは30秒後、60秒後、120秒後に各1回再試行します。すべて失敗すると停止し、consoleにエラーを記録します。トークン期限切れ後は、次のメッセージ送信が失敗し、既存のエラー通知が表示されます。会話と送信済みメッセージは消えません。

ユーザーが自社システムからログアウトするときは `MaiAgent.auth.signOut()` を呼び出します。SDKはIDを消去して更新を停止します。発行済みトークンは最長15分間有効な場合があります。別のユーザーがログインしたときは `MaiAgent.auth.setup()` を再度呼び出します。前のユーザーに対して進行中の更新は無効になります。

Web ChatでSource ID署名検証を有効にしている場合、署名は1回限りです。SDKは更新時に最初の `ts` `sig` を再利用しないため、`getToken` からトークンとともに新しい署名を返してください（`{ accessToken, ts, sig }`）。

`getToken` を使用せず、静的な `auth.accessToken` を直接指定した場合、SDKは更新しません。15分後に `MaiAgent.auth.setup()` を再度呼び出す必要があります。

2つの方式の選び方

	メンバーIDでMaiGPTを埋め込む（本節）	独自アシスタント+MaiGPTウィンドウモード（第2節、第3節）
利用者	組織メンバーである必要があります	誰でも利用できます（顧客、会員、匿名ユーザー）
機能	ロール権限に応じたMaiGPTの全機能：画像生成、Deep Research、コネクタ、スキル	選択したツールとナレッジベース
指示／モデルのカスタマイズ	できません（プラットフォームで一元管理）	できます
バックエンド作業	トークン発行APIを中継するエンドポイント1つ	不要（任意で連絡先IDの同期を追加）
課金先	メンバーのクォータ	アシスタント／Web Chat

6. よくある質問

- ▶ 管理画面の「会話プラットフォーム」にMaiGPTというWeb Chatがあります。その埋め込みコードを使えますか？
- ▶ MaiGPT製品の画像生成、Deep Research、コネクタは、独自アシスタントでも利用できますか？
- ▶ アシスタントをAgentモードにして画像生成ツールも追加しましたが、画像を生成しません。何を確認すべきですか？
- ▶ MaiGPTウィンドウモードとfloating／sidebarモードの違いは何ですか？

メンバーのメモリ管理 API

ログイン済みのメンバーは、`/api/me/memories/` を通じて自身の記憶を確認・管理できます。リクエストには `X-Organization-Id` を含め、現在操作する組織を指定してください。サービスは、その組織におけるログインユーザーの有効なメンバー資格に基づいてデータを取得します。ログイン方法と共通のリクエスト方法については、[クイックスタート](#)を参照してください。

利用可能な操作

操作	エンドポイント	動作
一覧表示	<code>GET /api/me/memories/</code>	現在のメンバーの <code>active</code> と <code>expired</code> の記憶を一覧表示します。 <code>status</code> と <code>kind</code> のクエリパラメータで絞り込みます。
作成	<code>POST /api/me/memories/</code>	自身の記憶を作成します。同じメンバーに同じ <code>topic</code> の有効な記憶がすでにある場合は、その記憶を更新します。新規作成時は HTTP 201、更新時は HTTP 200 を返します。
編集または再有効化	<code>PATCH /api/me/memories/{id}</code>	表示可能な記憶を編集するか、 <code>status</code> を <code>active</code> または <code>expired</code> に設定します。
削除	<code>DELETE /api/me/memories/{id}</code>	自身の指定した記憶を削除します。

作成時には `topic`、`summary`、`details` が必須です。`topic` は最大 100 文字で、小文字の英字、数字、および文字の間のアンダースコアのみ使用できます。`summary` は最大 500 文字です。`details` は最大 20 項目で、各項目は最大 500 文字です。`kind` には `fact`、`preference`、`instruction` を指定できます。省略した場合は `fact` になります。

一覧表示と個別操作では、`active` と `expired` の記憶のみが表示されます。確認待ちや拒否された記憶は、このメンバー向け画面には表示されません。

連携時の状態の扱い

メンバーが記憶を `expired` に設定しても、連携先は一覧で確認し、`PATCH` で `active` に戻せます。有効なメンバー資格がない場合、記憶の作成は HTTP 404 を返します。異なる組織のデータが同じ画面に混在しないよう、連携先ではリクエストで指定した組織に対応する記憶を表示してください。

ソフトウェアベンダー向け連携ガイド：顧客向けの AI Agent の埋め込み

このページは、MaiAgent Web Chat を自社製品に埋め込み、自社のアカウントシステムと権限連携するソフトウェアベンダーを対象としています。ユーザーが自社システムにログインすると、AI Agent はユーザーを識別し、会話履歴を参照し、**そのユーザーの権限**で自社システムの API を呼び出せます。

統合プロセスは 5 つの段階で構成されています。このページでは、各段階で行うことと実施順序を説明し、詳細については対応する技術ドキュメントにリンクします。

統合の全体像

```
flowchart LR
    subgraph ONCE ["初回のみ構築"]
        A["Web Chat を埋め込む"] --> B["既存ユーザーを一括プロビジョニング"]
    end
    subgraph DAILY ["日常運用 (アカウントのライフサイクル)"]
        C["新規ユーザー作成時に連絡先を作成"] --> D["ログイン時に Token を更新"]
        D --> E["Token 更新時に再同期"]
        E --> F["ログアウト時に認証情報を失効"]
    end
    ONCE --> DAILY
```

Mermaid 図表

このページでいうユーザーは貴社の顧客（MaiAgent の連絡先に対応）です。連携対象が貴社または顧客企業の従業員であり、MaiAgent の組織メンバーとして MaiGPT、ロール権限、Credits を利用させる場合は、[メンバープロビジョニング：SCIM／招待／SSO](#)を参照してください。

中心となる概念は **連絡先（Contact）** だけです。自社システムの各ユーザーは、MaiAgent の 1 つの連絡先に対応します。連絡先には、会話履歴、パーソナライズ属性、ナレッジベースの検索範囲、ツールの認証情報が関連付けられます。権限連携のすべての操作は、本質的には「自社ユーザー ↔ 連絡先」の対応関係とその認証情報を維持するものです。概念の詳細は、[連絡先（Contact）の概要と連携](#)を参照してください。

フロントエンドとバックエンドの役割

一連のプロセスにおいて、フロントエンドは「チャットウィンドウの読み込み」のみを担当します。Token または API Key を含むすべての呼び出しは、バックエンドから実行します。

```

flowchart LR
    subgraph FE["貴社のフロントエンド"]
        A["埋め込みコードで SDK を読み込む  
config に contactId を指定"]
        B["ログアウト時に signOut を呼び出す"]
    end
    subgraph BE["貴社のバック  
エンド"]
        C["ログイン／更新：POST setup-contact-credentials"]
        D["一括プロビジョニング：POST  
bulk-import"]
        E["データ同期：PATCH contacts"]
        F["ログアウト：DELETE ツール認証情報"]
    end
    subgraph MA["MaiAgent"]
        G["チャットウィンドウ  
chat.maiagent.ai"]
        H["API  
api.maiagent.ai"]
    end
    A --> G
    B --> G
    C --> H
    D --> H
    E --> H
    F --> H

```

Mermaid 図表

`setup-contact-credentials` は API Key が不要な公開エンドポイントです。`bulk-import`、`PATCH`、`DELETE` にはすべて `Authorization: Api-Key` 認証が必要であり、バックエンドからのみ呼び出す必要があります。

ステップ 1：Web Chat を埋め込む

最初に、埋め込む AI Agent を決定してください。画像生成、Web アクセス、ツール呼び出しなどの埋め込み機能は、埋め込み方法ではなく Web Chat の背後にあるアシスタント設定によって決まります。組織に組み込まれている「MaiGPT」製品はログイン済みメンバー専用であり、その Web Chat を外部システムに埋め込むことはできません。製品用の AI Agent を作成し、Agent モードに設定して必要なツールを追加してください。詳しくは、[MaiGPT の連携：適切なアシスタントの選択と必要な機能の追加](#)を参照してください。

製品の形態に応じて埋め込み方法を選択してください。どちらも同じ SDK と連絡先の仕組みを使用します。

形態	適した用途	ドキュメント
チャットバブル (floating／sidebar)	カスタマーサポートや問い合わせなどの補助機能	Web Chat の埋め込みと SDK
MaiGPT モード (ChatGPT に似たフルインターフェース)	ページのメイン機能領域、サイト全体の AI エントリーポイント	MaiGPT モードの埋め込み

ユーザー識別を連携する Web Chat では、この段階で「埋め込みを許可するドメイン」のリストを設定することを推奨します。現在は MaiAgent チームが設定するため、連携担当者にドメインリストを提供してください。これにより、ユーザー識別同期 API を呼び出せるオリジンを限定できます。詳しくは、[埋め込みを許可するドメイン](#)を参照してください。

ステップ 2：初回統合——既存ユーザーの一括プロビジョニング

既存システムを初めて連携する際は、既存ユーザーの連絡先を一括で作成します。次の 2 つの方法があります。

方法	適した用途	ドキュメント
Excel 一括インポート (<code>POST /contacts/bulk-import/</code>)	多数のユーザー (1 回につき最大 10,000 行)。ナレッジ検索範囲と API ツールの認証情報も同時に取り込みます	連絡先の一括インポート
ユーザーごとにユーザー識別同期 API を呼び出す	ログインフローに沿った段階的なプロビジョニング、または少数のユーザー	既存ユーザーの一括プロビジョニング

プロビジョニング時にユーザーの Access Token を取得できなくても問題ありません。まず対応関係だけを作成し、ステップ 4 のログインフローで、各ユーザーが次回ログインしたときに Token の認証情報を追加します。

ステップ 3：新規ユーザー作成時に連絡先を作成する

一括プロビジョニングの完了後、自社システムの「アカウント作成」フローに API 呼び出しを追加し、新規ユーザーに対応する連絡先がすぐに作成されるようにします。

バックエンドから ユーザー識別同期 API (`setup-contact-credentials`) を呼び出し、自社システムのユーザー識別子を `sourceId` として使用します。

返された `contactId` を会員データベースに保存します。推奨するフィールド設計については、[ログインフローへの統合位置](#)を参照してください。

`sourceId` は冪等性キーです。同じ `sourceId` で繰り返し呼び出しても重複作成されず、既存のレコードが更新されるため、このステップはステップ 2 およびステップ 4 と安全に併用できます。

ステップ 4：ログインと Token の更新

ログインが成功して Access Token が生成されるたびに、バックエンドから同じユーザー識別同期 API を再度呼び出し、**新しい Token** (`mcpCredentials`) を渡します。これにより、AI Agent はそのユーザーの識別情報と権限で自社システムの API を呼び出せます。Token の更新時も同じ方法で、もう一度 API を呼び出してください。同じ連絡先とツールの組み合わせは重複作成されず、更新されます。完全なシーケンス図と注意事項（非同期呼び出し、同期失敗でログインをブロックしない方法）については、[連絡先のユーザー識別同期と Token の更新](#)を参照してください。

フロントエンドでは、ページ上で Web Chat を初期化するときに `contactId` を指定します。詳しくは、[ユーザーの識別](#)を参照してください。

AI が自社システムの API を呼び出す必要がなく、ユーザーを識別するだけでよい場合は、バックエンドを使用せず、フロントエンドの config に auth を指定することもできます。選択時の考慮事項については、[フロントエンドの auth とバックエンド連携の選び方](#)を参照してください。

ステップ 5：ログアウトと認証情報の失効

ログアウト時には、作用範囲が異なる次の 2 つの操作を両方行うことを推奨します。

フロントエンド：SDK の `signOut()` を呼び出し、チャットウィンドウを匿名状態に戻します。

バックエンド：該当する連絡先のツール認証情報を削除し、AI がそのユーザーとしてツールを呼び出せないようにします。

API の詳細と 2 つの操作の違いについては、[ログアウトと認証情報の失効](#)を参照してください。

統合チェックリスト

段階	操作	ドキュメント
構築	AI Agent を作成：Agent モードを有効化し、画像生成などのツールを追加して Web Chat を作成	MaiGPT の連携：適切なアシスタントの選択と必要な機能の追加
構築	Web Chat（通常モードまたは MaiGPT モード）を埋め込む	Web Chat の埋め込みと SDK / MaiGPT モード
構築	MaiAgent にドメインリストを提供し、「埋め込みを許可するドメイン」を設定	埋め込みを許可するドメイン
構築	会員データベースに <code>maiaagent_contact_id</code> フィールドを追加	ログインフローへの統合位置
構築	既存ユーザーを一括プロビジョニング	連絡先の一括インポート
運用	アカウント作成フローでユーザー識別同期 API を呼び出す	連絡先のユーザー識別同期と Token の更新
運用	ログイン / Token 更新フローで認証情報を更新	呼び出すタイミング

段階	操作	ドキュメント
運用	ログアウトフローで認証情報を失効	<u>ログアウトと認証情報の失効</u>
応用	部署、会員ランクなどのパーソナライズ属性を同期	<u>連絡先データの同期</u>
応用	クエリメタデータでユーザーごとのナレッジベース検索範囲を制限	<u>ナレッジ管理権限（Query Metadata）の概要</u>

Web Chat の埋め込みと SDK

ウェブページに埋め込みコードを追加するだけで、MaiAgent の AI Agent をお客様のウェブサイト設置できます。埋め込みコードが `embed.min.js`（以下 SDK）を読み込むと、SDK はページ上にチャットボタンと iframe のチャットウィンドウを作成し、プログラムから操作できるグローバル `MaiAgent` JavaScript API を提供します。

本ページは Web Chat 埋め込みの完全な技術リファレンスです。関連トピック：

[MaiGPT 連携：適切なアシスタントの選択と機能の補完](#) — MaiGPT 製品と MaiGPT ウィンドウモードの違い、および埋め込んだアシスタントで画像生成やツール利用を可能にする方法

[Web Chat MaiGPT モード埋め込み](#) — ChatGPT ライクなフル機能の会話インターフェース

[WebChat ページ Context の注入](#) — ページ情報を LLM System Prompt に注入

[連絡先の身元同期と Token 更新](#) — バックエンドで連絡先と資格情報を作成

[ソフトウェアベンダー連携ガイド](#) — 埋め込み+権限連携の完全なジャーニー（バッチ補完作成、ログイン同期、ログアウト失効）

一、クイックスタート

ページの `<body>` の前に次を追加します：

```
<script>
  window.maiagentChatbotConfig = {
    webChatId: 'your-web-chat-id',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    primaryColor: '#1890ff',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`webChatId` は管理コンソールの Web Chat 設定ページから取得します（[チャットプラットフォーム連携：ウェブサイト](#)を参照。コンソールが生成する埋め込みコードには自動で設定されます）

`baseUrl` と SDK の URL は SaaS 環境（`chat.maiagent.ai`）を例にしています。プライベートクラウド／オンプレミスの場合は、お客様の環境のドメインに置き換えてください

`window.maiagentChatbotConfig` は必ず SDK スクリプトの読み込み前に定義してください

デフォルトの動作：ページ右下にチャットボタンが表示され、クリックするとフローティングチャットウィンドウが開きます。ユーザーはフローティング（floating）とサイドバー（sidebar）の2つのウィンドウモードを切り替えられます。

初期化のタイミング

SDK は読み込み後に `document.body.onload` ハンドラを登録して初期化を実行するため、通常は手動での介入は不要です。初期化のタイミングを制御したい場合（SPA のコンテナ mount 待ちや遅延レンダリングなど）：

スクリプトの遅延読み込み：適切なタイミングで `embed.min.js` の